

Content-Based Filtering

This approach differs from collaborative filtering by explicitly using the attributes (features) of users and items to find a good match.

Collaborative vs. Content-Based

- **Collaborative Filtering:** Recommends items based on the behavior of *similar users* (e.g., "People who bought X also bought Y"). It does not need to know *what* the item is.
 - **Content-Based Filtering:** Recommends items based on the match between a user's *profile* and an item's *features*. It requires specific features for both.
-

Feature Vectors

1. User Features ($x_u^{(j)}$)

We construct a feature vector for user j based on what we know about them.

- **Demographics:** Age, Gender (one-hot), Country (one-hot).
- **Behavior:** Which of the top 1,000 movies have they watched?
- **Derived Preferences:** What is their average rating for "Romance" movies? What is their average rating for "Action" movies?
 - *Note:* It is perfectly fine to build user features based on their past ratings.

2. Movie (Item) Features ($x_m^{(i)}$)

We construct a feature vector for movie i .

- **Attributes:** Release year, Genre (one-hot), Cast, Studio.
- **Reviews:** Features derived from critic reviews.
- **Derived Stats:** Average rating of this movie.

Key Difference: The user feature vector (x_u) and the movie feature vector (x_m) can have **completely different sizes**. (e.g., user vector length = 1500, movie vector length = 50).

The Matching Concept: Dot Products

In Collaborative Filtering, we predicted a rating as $w^{(j)} \cdot x^{(i)} + b$.

In Content-Based Filtering, we want to predict the rating by taking the **dot product of two vectors**: a "User Vector" (v_u) and a "Movie Vector" (v_m).

$$\text{Prediction} = v_u^{(j)} \cdot v_m^{(i)}$$

- $v_u^{(j)}$: A vector representing the user's preferences (computed from user features x_u).
- $v_m^{(i)}$: A vector representing the movie's attributes (computed from movie features x_m).

Crucial Requirement:

Even though the raw inputs x_u and x_m have different sizes, the computed vectors v_u and v_m **must be the same size** (e.g., both are length 32) so we can take their dot product.

Example Interpretation:

- If $v_u = [4.9, 0.1]$ (User loves Romance, hates Action).
 - If $v_m = [4.5, 0.2]$ (Movie is highly Romance, low Action).
 - The dot product will be high, indicating a good match.
-

The Goal

The challenge of Content-Based Filtering is: **How do we turn the raw features (x_u, x_m) into these compact, matching vectors (v_u, v_m)?**

Deep Learning for Content-Based Filtering 🧠

To implement content-based filtering effectively, we use Neural Networks to transform the raw features of users and items into vectors that can be compared mathematically.

The Architecture: Two Towers

We build two separate neural networks that work together.

1. The User Network

- **Input:** User features vector x_u (Age, Gender, Country, Past behavior, etc.).
- **Hidden Layers:** Dense layers to process the data.
- **Output:** A user vector v_u (also called an **embedding**).
 - *Example Size:* A vector of 32 numbers.

2. The Movie (Item) Network

- **Input:** Movie features vector x_m (Year, Stars, Genre, etc.).
- **Hidden Layers:** Dense layers to process the data.
- **Output:** A movie vector v_m .
 - *Constraint:* This output vector **must** be the same size as the user vector v_u (e.g., 32 numbers).

***Note:** The two networks can have completely different structures (different numbers of hidden layers or node counts) internally. They only need to match at the **final output layer**.*

Making Predictions

Once the networks compute v_u and v_m , we combine them to make a prediction.

A. For Regression (Predicting Star Ratings):

We take the **Dot Product** of the two vectors.

$$\text{Prediction} = v_u \cdot v_m$$

B. For Binary Classification (Predicting Probability of Like/Click):

We apply the **Sigmoid** function to the dot product.

$$\text{Probability} = g(v_u \cdot v_m)$$

Training the Model

We train both networks simultaneously as a single combined model.

The Cost Function (J):

We want to minimize the difference between the predicted rating (dot product) and the actual rating (y).

$$J = \sum_{(i,j):r(i,j)=1} (v_u^{(j)} \cdot v_m^{(i)} - y^{(i,j)})^2 + \text{Regularization}$$

- **Process:** We use gradient descent. The error from the cost function backpropagates through the dot product, updating the weights in **both** the User Network and the Movie Network at the same time.
 - **Goal:** The networks learn to generate vectors v_u and v_m such that their dot product accurately approximates the user's rating.
-

Finding Similar Items

A powerful side-effect of this architecture is that the Movie Network learns a vector representation (v_m) for every movie. We can use this to find related items.

- To find movies similar to Movie i :
 1. Take the vector $v_m^{(i)}$.
 2. Search for other movies k where the distance $\|v_m^{(k)} - v_m^{(i)}\|^2$ is small.
- **Practical Tip:** Since this calculation only depends on movie features (not users), it can be **pre-computed** (e.g., run overnight) and stored. When a user looks at a movie page, the system can instantly look up the pre-calculated "Similar Movies."

Summary

- **Pros:** Very powerful; can combine complex features (text, images, demographics) using neural networks.
- **Cons:** Computationally expensive to run for every user on a massive catalog of items in real-time.

Scaling Recommender Systems: Retrieval and Ranking 🚀

When you have a catalog of **millions** of items (songs, videos, products), it is computationally impossible to run a complex neural network prediction for every single item every time a user visits your site.

To solve this, large-scale systems split the process into two steps: **Retrieval** and **Ranking**.

Step 1: Retrieval (Candidate Generation)

The goal of this step is fast, broad coverage. We want to quickly narrow down millions of items to a manageable list of candidates (e.g., hundreds) that contain at least *some* items the user will like. It is okay if this list includes some bad recommendations, as they will be filtered out later.

Common Retrieval Strategies:

- **Similar to Recent History:** For the last 10 items the user engaged with, look up the "most similar items" (using pre-computed movie vectors v_m).
- **Top Genres:** If the user loves "Sci-Fi," retrieve the top 10 most popular Sci-Fi movies.
- **Demographics:** Retrieve the top 20 most popular movies in the user's country.

Result: A list of perhaps 500–1000 plausible candidates. (Duplicates and items already watched are removed).

Step 2: Ranking

The goal of this step is accuracy. We take the small list of candidates from the retrieval step and use our powerful Neural Network model to rank them.

The Process:

1. **Compute User Vector:** Run the user's features through the User Network to get v_u . (Done once).
2. **Predict Ratings:** For each of the ~500 candidates, perform the inference (dot product $v_u \cdot v_m$) to predict the user's specific rating or probability of clicking.
3. **Sort:** Order the items by their predicted score.
4. **Display:** Show the top 10–20 items to the user.

[Image of anomaly detection plot]

Performance Tuning

A key decision in this architecture is **how many items to retrieve** in Step 1.

- **Retrieving More Items (e.g., 1000):**
 - *Pros:* Better chance of finding the absolute best content (higher accuracy).
 - *Cons:* Slower system performance (higher latency).
- **Retrieving Fewer Items (e.g., 100):**
 - *Pros:* Very fast.
 - *Cons:* Risk of missing the best content because it was never retrieved for ranking.

Recommendation: Perform offline experiments. Check if increasing the retrieval count significantly improves the quality of the final top-10 list. If the improvement is small, stick to a smaller number for speed.

Summary of Recommender Systems Architecture

1. **Catalog:** Millions of items.
2. **Retrieval:** Fast heuristics filter this down to ~500 candidates.
3. **Ranking:** Neural Networks predict precise scores for those ~500.
4. **Output:** Top 10 items shown to the user.

The Ethics of Recommender Systems

Recommender systems are incredibly profitable, but they have significant power to influence user behavior and society. While many use cases are benign, specific design choices can lead to unintended, harmful consequences.

1. Choosing the System's Goal

The most critical decision an engineer makes is defining **what the system should optimize for**.

- **User-Centric Goals (Generally Benign):**
 - Recommend items the user will rate 5 stars.
 - Recommend products the user is most likely to purchase.
 - *Result:* The system tries to maximize value/enjoyment for the user.
 - **Company-Centric Goals (Potentially Problematic):**
 - **Maximize Ad Revenue:** Show ads likely to be clicked *and* where the advertiser pays the most.
 - **Maximize Profit:** Recommend products with the highest profit margin (buying cheap, selling high) rather than the best product for the user.
 - **Maximize Engagement/Watch Time:** Keep the user on the site for as long as possible to show more ads.
-

2. The Problem of Ad Amplification

Recommender systems generally amplify businesses that are profitable (because profitable businesses can bid more for ads). This creates a positive feedback loop that can be virtuous or vicious.

- **The Virtuous Cycle (e.g., A Great Travel Agency):**
 - Company provides great service → Becomes profitable → Bids high on ads → Gets more traffic → Serves more happy users.
- **The Vicious Cycle (e.g., Predatory Payday Loans):**
 - Company exploits low-income users → Becomes highly profitable (by squeezing customers) → Bids high on ads → Gets more traffic → Exploits more people.

The Solution? Hard to implement, but platforms may need to refuse ads from exploitative industries. Defining "exploitative" is difficult but necessary.

3. The Problem of Toxic Engagement 🦴

When a system is optimized strictly for "**Maximum Watch Time**" or "**Maximum Engagement**," it often discovers a dark truth about human psychology:

- **Conspiracy theories, hate speech, and toxic content** are often highly engaging.
- The algorithm, blindly chasing engagement numbers, begins to push this harmful content to users.

The Solution:

- **Filtering:** Systems must actively filter out hate speech, fraud, scams, and violence.
 - **Metric Adjustment:** Engineers should consider optimizing for "quality time" or "user satisfaction" rather than just raw duration.
-

4. The Transparency Gap 🔍

There is often a disconnect between what the user *thinks* the system is doing and what it is *actually* doing.

- **User Expectation:** "This app is recommending this movie because it thinks I will enjoy it."
- **Reality:** "This app is recommending this movie because the studio paid to promote it," or "This app is showing this product because it has a high profit margin."

The Solution: Be transparent. Let users know the criteria used to make recommendations. This builds trust.

Summary: A Call to Action

Technology is not neutral. If you build these systems, you must:

1. **Look beyond the metric:** Don't just look at profit or engagement; consider the societal impact.
2. **Invite diverse perspectives:** Debate and discuss potential harms during the design phase.
3. **Prioritize well-being:** Only build things that you genuinely believe make people and society better off.

Implementing Content-Based Filtering in TensorFlow

This video walks through the specific code structure for building the "Two-Tower" neural network architecture using the TensorFlow Keras API.

1. Defining the Networks

We create two separate Sequential models: one for the user and one for the item (movie). Both must end with an output layer of the same size (e.g., 32 units).

The User Network:

```
1 user_NN = tf.keras.models.Sequential([
2     # Hidden layers with Relu activation
3     tf.keras.layers.Dense(256, activation='relu'),
4     tf.keras.layers.Dense(128, activation='relu'),
5     # Output layer (the "User Vector")
6     tf.keras.layers.Dense(32)
7 ])
```

The Item (Movie) Network:

```
1 item_NN = tf.keras.models.Sequential([
2     # Hidden layers with Relu activation
3     tf.keras.layers.Dense(256, activation='relu'),
4     tf.keras.layers.Dense(128, activation='relu'),
5     # Output layer (the "Item Vector")
6     tf.keras.layers.Dense(32)
7 ])
```

2. The Forward Pass and Normalization

We define the input layers, pass them through the networks to get the vectors (v_u and v_m), and then apply a crucial normalization step.

Why Normalize?

Normalizing the vectors to have a length of 1 (L2 norm) significantly improves the algorithm's performance.

```
1 # 1. Create Inputs
2 input_user = tf.keras.layers.Input(shape=(num_user_features))
3 input_item = tf.keras.layers.Input(shape=(num_item_features))
4
5 # 2. Compute Vectors (Pass inputs through the networks)
6 vu = user_NN(input_user)
7 vm = item_NN(input_item)
8
9 # 3. L2 Normalization (Scale vectors to length 1)
10 vu = tf.linalg.l2_normalize(vu, axis=1)
11 vm = tf.linalg.l2_normalize(vm, axis=1)
```

3. The Dot Product and Model Definition

Finally, we calculate the predicted rating by taking the dot product of the two normalized vectors.

```
1 # 4. Compute Dot Product (The Prediction)
2 # Keras has a specific layer for this operation
3 output = tf.keras.layers.Dot(axes=1)([vu, vm])
4
5 # 5. Define the Full Model
6 # Inputs: User features AND Item features
7 # Output: The dot product prediction
8 model = tf.keras.Model([input_user, input_item], output)
9
10 # 6. Compile the Model
11 # Use Mean Squared Error cost function
12 model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
13               loss='mean_squared_error')
```

Summary of Course 3, Week 2

You have now completed the deep dive into Recommender Systems!

- **Collaborative Filtering:** Recommendations based on user/item interactions (ratings).
- **Content-Based Filtering:** Recommendations based on user/item features using Neural Networks.
- **Architecture:** Retrieval (fast candidate generation) → Ranking (precise scoring).
- **Ethics:** The importance of optimizing for user well-being, not just profit or engagement.